# Clinical Consultation Copilot

Chronological Operational Specification: Ingestion to Downstream LLM Synthesis

**Date:** July 04, 2026 | **Operational Status:** Fully Verified | **Process Pipeline:** Step 1 to Step 4 ⇒ LLM

## Phase 1: Session Handshake & Connection Initialization

The operational lifecycle begins at the client application edge. When a clinician begins a new consultation session, the client application (Web or Mobile) establishes a connection to the backend:

- **Connection Handshake:** The client initiates a secure full-duplex WebSocket connection to the backend gateway at wss://[host]/v1/streaming/audio.
- **Query Parameter Authentication:** The handshake contains three critical query parameters: session_id (a unique UUID representing the consultation), role (identifying the participant as doctor, patient, or attender), and a secure token.
- **Active Registry Binding:** The SessionManager validates the JWT token, verifies that the session is active, and registers the WebSocket connection, binding it to the specific participant role in-memory.

## Phase 2: Real-Time Audio Streaming & In-Memory Buffering

Once authenticated, the WebSocket connection transitions into a continuous data streaming mode:

- **Raw Audio Capture:** The client captures audio in a raw, uncompressed format: **Linear PCM (LPCM) 16-bit, 16kHz, Mono**. This ensures maximum phonetic clarity for the transcription engine and bypasses compression CPU overhead on the device.
- **Micro-Frame Streaming:** The client streams the audio in tiny slices (20ms to 50ms per packet; i.e., 640 to 1600 bytes) to maintain an ultra-low local capture latency and avoid network queue delays.
- **On-the-Fly Byte Validation:** On the backend, the AudioFrameValidator inspects each incoming binary packet to ensure it adheres to a 2-byte sample boundary and falls within the expected size constraints, rejecting corrupted frames.
- **Stable Chunk Accumulation:** The AudioSessionBuffer accumulates these tiny micro-frames in a high-speed in-memory bytearray. When the buffer reaches exactly **1.0 second of audio (32,000 bytes)**, it packages the chunk and pushes it to a role-specific asyncio.Queue.
- **Memory Protection & Backpressure:** Each stream is capped at a strict **10MB memory threshold**. If network congestion or processing delays block the queue, the buffer triggers backpressure policies (e.g., drop_oldest) to prevent RAM exhaustion.

## Phase 3: Real-Time Speech-to-Text (STT) Pipeline

Once a stable 1.0-second audio chunk is queued, the transcription layer processes it asynchronously:

- **Stateless STT Reconciliation Loop:** A background STTManager runs a continuous 100ms loop, monitoring the active session registry. It dynamically spawns dedicated background worker tasks for each newly connected role and tears them down when the connection closes.
- **WebSocket Streaming to STT Provider:** The spawned worker task reads the 1.0s chunks from the queue and streams them over a persistent WebSocket connection to a high-accuracy speech-to-text service (such as ElevenLabs Scribe V2 or Whisper).
- **Self-Healing Connections:** The worker task implements a robust **120-second timeout guard**. If the STT provider's socket hangs or disconnects due to network jitter, the worker automatically re-establishes the socket connection and flushes the local buffer, preventing conversational data loss.
- **Transcript Bus Dispatch:** As transcription results are returned, they are parsed and published to a thread-safe, in-memory TranscriptBus, making the text events available to downstream processors.

## Phase 4: Speaker Diarization & Temporal Alignment

Transcription alone is insufficient for multi-party consultations; the system must attribute text to the correct speakers:

- **In-Memory Tensor Diarization:** The diarization_worker processes the raw audio stream. It normalizes peak amplitudes in-memory and converts the raw LPCM bytes directly into numerical PyTorch tensors, feeding them into the pyannote/speaker-diarization-3.1 model (running on CUDA/CPU) without writing slow temporary files to disk.
- **4-Tier Temporal Cascade:** Due to the acoustic complexity of clinical environments (e.g., interruptions, doctor and patient speaking at once), the SpeakerTimeline engine resolves overlapping speech using a strict 4-tier assignment cascade:
  1. *Overlap Resolution:* Determines dominant speaker energy during overlapping segments.
  2. *Proximity Assignment:* Attributes unassigned words to the nearest active speaker segment.
  3. *Continuity Mapping:* Bridges short pauses to maintain conversational flow.
  4. *Hard Caps:* Prevents single-speaker turns from extending past maximum reasonable limits.
- **Word-Level Timestamp Alignment:** The engine aligns individual words from the STT stream with the physical diarized speaker boundaries, producing a chronologically aligned chronological transcript (e.g., attributing word arrays to the doctor, patient, or attender).
- **Utterance Merging & Filler Stripping:** Consecutive turns by the same speaker are merged into a single dialogue block, and conversational filler words (e.g., *'uh'*, *'um'*, *'like'*) are stripped out by the FillerRemover.

## Phase 5: Local Clinical Fact Extraction & Rules Engine

The clean, aligned dialogue turns are immediately pushed to the ClinicalProcessingPipeline. This layer executes a high-speed, local rules engine to extract structured medical concepts:

- **Clinician Question Suppression:** Before extracting findings, the engine inspects the speaker ID. If the speaker is the doctor and the utterance is classified as a question (e.g., *'Do you have a fever?'*), the engine suppresses the turn. This ensures that clinical queries do not register as active patient symptoms.
- **Layman Term Normalization:** The ClinicalNormalizer standardizes colloquial patient descriptions to clinical concepts (e.g., translating *'head ache'* or *'throbbing head'* to *'headache'*, and *'watery pills'* to *'amlodipine'*).
- **Atomic Entity Extraction:** Programmatic regular expression patterns match terms against six clinical categories: Symptoms, Medications, Diagnoses, Procedures, Risk Factors, and Family Histories.
- **Clause-Isolated Negation Resolution:** The text is split into clauses. The engine evaluates pre-negation triggers (e.g., *'denies'*, *'no'*, *'without'*) and post-negation triggers (e.g., *'resolved'*, *'ruled out'*). It correctly identifies double-negations (e.g., *'Fever was not ruled out today'* resolves to present: true).

## Phase 6: Session-Level State Aggregation & Deduplication

To compile a cohesive summary, individual turn extractions are continuously aggregated into a session-level state database:

- **Incremental Deduplication:** When a symptom or medication is extracted multiple times across a session, the aggregator deduplicates it, preventing redundant entries in the clinical record.
- **Attribute Reconcilation:** The aggregator merges attributes from different turns. If the patient mentions a *'headache'* early in the session, and later specifies it is *'severe'* and has lasted *'two weeks'*, the engine updates the existing headache entity with the new severity and duration.

### Aggregated Session State Output Payload (Example):


```
{
  "session_id": "consultation-session-001",
  "symptoms": [
    { "name": "headache", "severity": "severe", "duration": "two weeks", "present": true, "confidence": 1.0 },
    { "name": "dizziness", "severity": null, "duration": null, "present": true, "confidence": 1.0 },
    { "name": "fever", "severity": null, "duration": null, "present": false, "confidence": 1.0 }
  ],
  "medications": [
    { "name": "paracetamol", "present": true, "confidence": 1.0 },
    { "name": "amlodipine", "present": true, "confidence": 1.0 }
  ],
  "diagnoses": [
    { "name": "hypertension", "present": true, "confidence": 1.0 },
    { "name": "migraine", "present": true, "confidence": 1.0 }
  ]
}
```

## Phase 7: Context-Driven Downstream LLM Triggering (The End)

The final phase of the operational workflow is the transition from Tier 1 (Local Rule Engine) to Tier 2 (LLM Engine). The system evaluates the context accumulated in the session-level state, triggering a downstream LLM call only when one of these five specific triggering conditions is met:

| Trigger Condition | Threshold Criteria & Asynchronous LLM Action |
|---|---|
| **1. Session End / Flush** | **Trigger Criteria:** The clinician explicitly clicks 'End Consultation', 'Generate Note', or the WebSocket connection closes.<br>**LLM Action:** Receives the full dialogue transcript and the structured state payload to synthesize the final clinical note (EHR or SOAP format), formatting it into a professional layout. |
| **2. Section Transition** | **Trigger Criteria:** The local parser detects a conversational transition between major clinical sections (e.g., from subjective HPI to objective physical exam).<br>**LLM Action:** Generates a targeted, sectional summary in the background, updating the draft EHR note incrementally. |
| **3. Inactivity Pause** | **Trigger Criteria:** The audio streaming layer detects a continuous silence period of 10 to 15 seconds across all active channels (e.g., doctor examining patient).<br>**LLM Action:** Runs an intermediate background summary task, catching up on the consultation draft without interrupting the active conversation flow. |
| **4. Semantic Density** | **Trigger Criteria:** The local aggregator collects a density of 5+ new clinical findings or 10-15 conversational dialogue turns.<br>**LLM Action:** Evaluates the new segment, updates the global state, and reconciles any contradictions or patient self-corrections. |
| **5. Safety Alert (Override)** | **Trigger Criteria:** The local rules engine extracts a critical clinical emergency (e.g., 'chest pain', 'anaphylaxis') or a dangerous drug-drug interaction.<br>**LLM Action:** Bypasses all buffering queues, performs an immediate safety audit, and flashes a high-priority warning directly onto the clinician's screen. |

## System Conclusion & Operational Validation

By structuring the backend into a chronological, 7-phase operational lifecycle, the Clinical Consultation Copilot succeeds in balancing high-performance, real-time interactivity with intelligent, context-driven synthesis. The Tier 1 local rule engine handles immediate, low-latency, deterministic filtering and extraction, while the Tier 2 LLM engine is triggered selectively when sufficient conversational context has accumulated. This hybrid pipeline minimizes API latency, reduces operational costs by up to 90%, and enforces strict clinical safety standards.